Лабораторная работа 1. Задание 3: блокировка чтения-записи

1. Постановка задачи

Сделать свою блокировку чтения-записи с функциями `rdlock` и `wrlock` и проверить её на отсортированном односвязном списке из `pth_ll_rwl.c`. Та же программа собирается второй раз с библиотечной `pthread_rwlock_t`, времена сравниваются.

Читателей может быть несколько сразу. Писатель только один, и только когда читателей нет.

Запуск:

```
./ll_my  <потоки> [ключей_в_main операций доля_поиска доля_вставки]
./ll_lib <потоки> [ключей_в_main операций доля_поиска доля_вставки]
```

`<потоки>` — число потоков. `ключей_в_main` — сколько ключей вставить в список до замера. `операций` — сколько всего поисков, вставок и удалений сделать. `доля_поиска` и `доля_вставки` — какая часть операций приходится на поиск и на вставку. Без четырёх последних чисел программа спрашивает их с клавиатуры, как исходный файл курса. Доля удалений равна `1 − поиск − вставка`.

Пример только поиска: `./ll_my 4 8000 12000 1.0 0`. Пример смеси: `./ll_my 4 1000 24000 0.90 0.01`. У `./ll_lib` аргументы те же.

2. Сведения о реализованном алгоритме

Структура блокировки — из пункта 2.3.2 методички: мьютекс, две условные переменные (для читателей и для писателей), счётчик текущих читателей, счётчики ожидающих чтение и запись, флаг писателя.

Мьютекс защищает только эти поля. Обход списка идёт после того, как `rdlock` или `wrlock` отпустил мьютекс. Иначе читатели стояли бы в очереди на мьютексе, и параллельного чтения не было бы.

Правила:

- Новый читатель ждёт, если писатель уже внутри или хотя бы один писатель стоит в очереди. Иначе сплошной поток поисков не подпустит вставку.
- Писатель ждёт, пока читателей нет и флаг писателя сброшен.
- `pthread_cond_wait` стоит в цикле `while` и проверяет само условие. Условная переменная может разбудить поток без причины, при этом `wait` возвращает 0 и на время сна сам отпускает мьютекс.
- `unlock` будит кого-то, только когда внутри уже никого нет. Если в очереди есть писатель, его будит `signal`. Иначе читателей поднимает `broadcast`.

Поиск по списку берёт `rdlock`, вставка и удаление — `wrlock`. Сами `Insert`, `Member` и `Delete` не менялись. Случайные ключи даёт `my_rand`: у каждого потока свой `seed`, общего состояния нет.

3. Пояснения к коду

Своя блокировка — `my_rwlock.c` и `my_rwlock.h`. Список и замер — `pth_ll_rwl.c`. Генератор и секундомер — `my_rand.c` и `timer.h` из материалов курса. Две сборки:

```
gcc -O2 -pthread -DUSE_MY_RWLOCK -o ll_my pth_ll_rwl.c my_rwlock.c my_rand.c
gcc -O2 -pthread -o ll_lib pth_ll_rwl.c my_rand.c
```

Флаг `-DUSE_MY_RWLOCK` подставляет свою реализацию вместо `pthread_rwlock_t`. Остальной код один и тот же.

`rdlock`. Захватывает мьютекс, увеличивает число ожидающих читателей и ждёт, пока нет писателя внутри и нет ожидающих писателей. Потом уменьшает очередь, увеличивает число текущих читателей и отпускает мьютекс.

`wrlock`. Так же ставит себя в очередь писателей и ждёт, пока нет ни текущего писателя, ни читателей. Затем ставит флаг писателя и отпускает мьютекс.

`unlock`. Если флаг писателя стоял, сбрасывает его, иначе уменьшает число читателей. При нуле читателей и ненулевой очереди писателей будит одного писателя. Если писателей в очереди нет, а читатели есть, будит всех читателей.

Поток `Thread_work`. Делает `total_ops / thread_count` операций. Доля `my_drand` выбирает поиск, вставку или удаление. Счётчики операций копятся в частных переменных и один раз в конце складываются под отдельным `count_mutex`. К списку этот мьютекс не относится.

Функция `main`. До `pthread_create` вставляет начальные ключи. Время берётся макросом `GET_TIME` вокруг создания потоков и `pthread_join`. После `join` проверяется, что список строго возрастает и без повторов.

Вывод программы: на экран — время в секундах и сколько было поисков, вставок и удалений. При ошибке порядка в списке программа завершается с кодом 1.

4. Условия измерений

- Процессор: Intel Core i5-13420H. В WSL он виден как 6 ядер и 12 потоков.
- Система: Ubuntu 22.04.5, ядро 6.6.87.1-microsoft-standard-WSL2, памяти выделено 3.7 ГБ. Сборка: gcc 11.4.0, флаги `-O2 -pthread`.
- Время — макрос `GET_TIME` из `timer.h` (`gettimeofday`) вокруг `pthread_create` и `pthread_join`. Начальные ключи в замер не входят.
- Каждая пара «реализация, число потоков» запускалась 5 раз, в таблицу положен минимум. Ускорение $S = T_1 / T_p$ считается отдельно для своей блокировки и для библиотечной: на одном потоке у них разное время.
- Число операций делится на число потоков нацело, поэтому суммарный объём работы при разном p один и тот же. При одинаковых аргументах обе программы делают одинаковое число поисков, вставок и удалений: последовательность задаёт `my_rand`, а не блокировка.

Две нагрузки:

- только поиск: 8000 ключей, 12 000 операций, доля поиска 1;
- смесь: 1000 ключей, 24 000 операций, поиск 0.90, вставка 0.01, удаление 0.09.

5. Результаты замеров

Только поиск, 8000 ключей, 12 000 операций:

Потоки p	Своя, с	S	pthread_rwlock, с	S
1	0.6791	1.00	0.7265	1.00
2	0.3123	2.17	0.3395	2.14
4	0.1788	3.80	0.1693	4.29
6	0.1497	4.54	0.1306	5.56
8	0.1108	6.13	0.1136	6.39
12	0.1150	5.90	0.1277	5.69

У библиотечной блокировки минимум на 4 потоках даёт S = 4.29. Среднее тех же пяти запусков — 0.196 с против 0.766 с на одном потоке, S = 3.91. Запас над четырьмя — разброс минимума.

Смесь, 1000 ключей, 24 000 операций, 90% поиск, 1% вставка, 9% удаление:

Потоки p	Своя, с	S	pthread_rwlock, с	S
1	0.0547	1.00	0.0500	1.00
2	0.1234	0.44	0.2023	0.25
4	0.3242	0.17	0.2282	0.22
6	0.5182	0.11	0.2322	0.22
8	0.5791	0.09	0.2268	0.22
12	0.6693	0.08	0.2460	0.20

Список после каждого прогона остаётся отсортированным.

6. Выводы

На чистом поиске своя блокировка делает то, что от неё требуется: несколько потоков читают список одновременно, цикл ожидания не засыпает, долгий обход идёт без мьютекса. Ближе всего к библиотечной она на 2 и на 8 потоках: ускорение 2.17 против 2.14 и 6.13 против 6.39. На 6 потоках библиотечная впереди заметнее, 5.56 против 4.54. До 8 потоков время ещё падает (своя: 0.150 с на 6 потоках и 0.111 с на 8). На 12 потоках минимум уже чуть хуже, чем на 8. В WSL процессор виден как 6 ядер по 2 потока, поэтому двенадцать потоков — это два логических потока на ядро.

На смеси с удалением дополнительные потоки время не уменьшают. Вставка и удаление берут исключительную блокировку, а пока писатель ждёт, новые читатели тоже стоят. Список короткий, обход дешевле очереди на мьютексе. Библиотечная блокировка после двух потоков держится около 0.23 с. Своя растёт и дальше, до 0.67 с на 12 потоках: приоритет писателя и один общий мьютекс на каждое взятие и отпуск. На одном потоке обе укладываются в 0.05 с, ждать некого.